# UKF with 2D State Vector

July 27, 2026

## 1 Design

### 1.1 State Vector

$$\mathbf{x}_t = \begin{bmatrix} z \\ \dot{z} \end{bmatrix} \tag{1}$$

### 1.2 Prediction Step

#### 1.2.1 Control Input

$$\mathbf{u}_t = \begin{bmatrix} \ddot{z} \end{bmatrix} \tag{2}$$

#### 1.2.2 State Transition Function

$$g(\mathbf{x}_{t-\Delta t}, \mathbf{u}_t, \Delta t) = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \mathbf{x}_{t-\Delta t} + \begin{bmatrix} \frac{1}{2}(\Delta t)^2 \\ \Delta t \end{bmatrix} \mathbf{u}_t \tag{3}$$

### 1.3 Measurement Update Step

#### 1.3.1 Measurement Vector

$$\mathbf{z}_t = \begin{bmatrix} r \end{bmatrix} \tag{4}$$

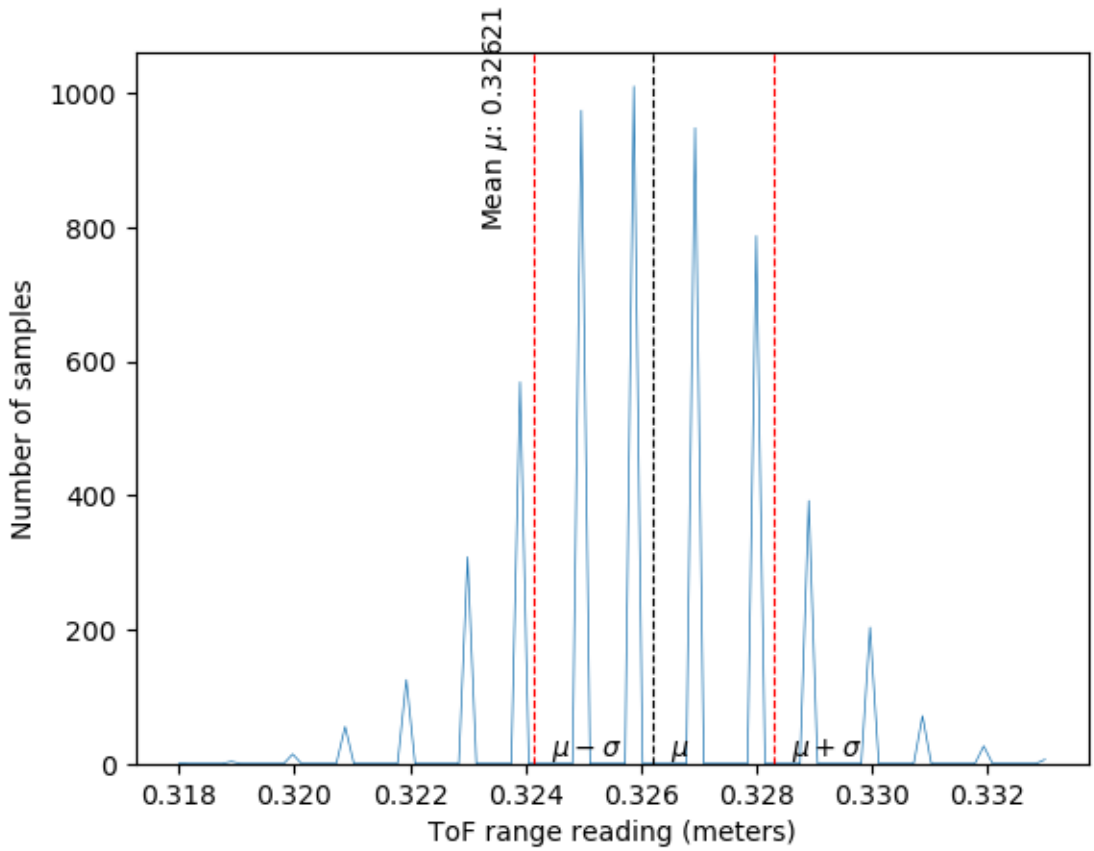
#### 1.3.2 Measurement Function

$$h(\bar{\mathbf{x}}_t) = \begin{bmatrix} 1 & 0 \end{bmatrix} \bar{\mathbf{x}}_t \tag{5}$$

#### 1.3.3 Measurement Covariance Matrix

Populate the measurement covariance matrix $\mathbf{R}_t$ with a reasonable value for the variance $\sigma_r^2$ of the range reading $r$.

$$\mathbf{R}_t = \begin{bmatrix} 4.325768932713897\text{e}{-06} \end{bmatrix} \tag{6}$$




# 2 Implementation

## 2.1 Initializing $\dot{z}$

To initialize $\dot{z}$, you could collect 2 readings before initializing the state. Then you could estimate the initial velocity by subtracting the first measurement from the second measurement and divide by the interval between them. Although it involves waiting for multiple measurements to initialize the filter, this method would give a more accurate estimate of the drone's initial state than just setting $\dot{z} = 0$ if the filter is launched while the drone is in motion. In practice, the filter will be launched before takeoff, when the drone is stationary on the ground. In this case, the drone's state has an initial $\dot{z} = 0$, and any nonzero initial velocity calculated would be due to errors/noise in the sensor readings. When the filter is initialized on a stationary drone, it makes more sense to initialize the state with $\dot{z} = 0$.

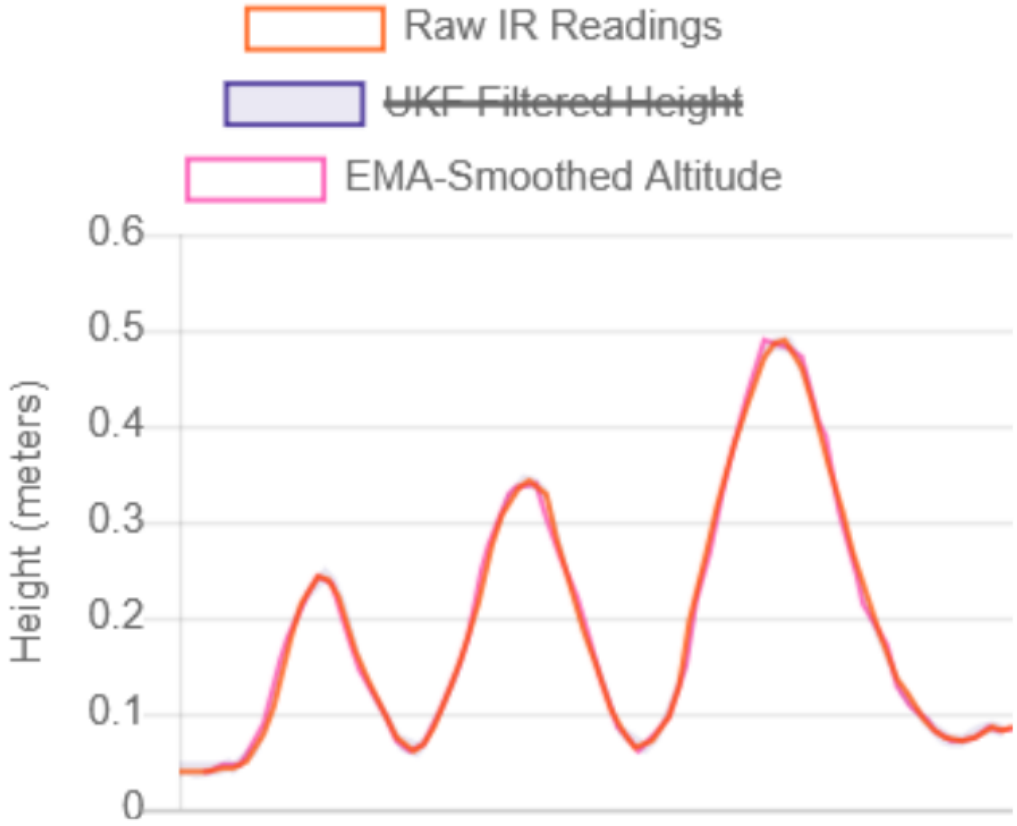
## 2.2 Consecutive Predictions vs. Updates

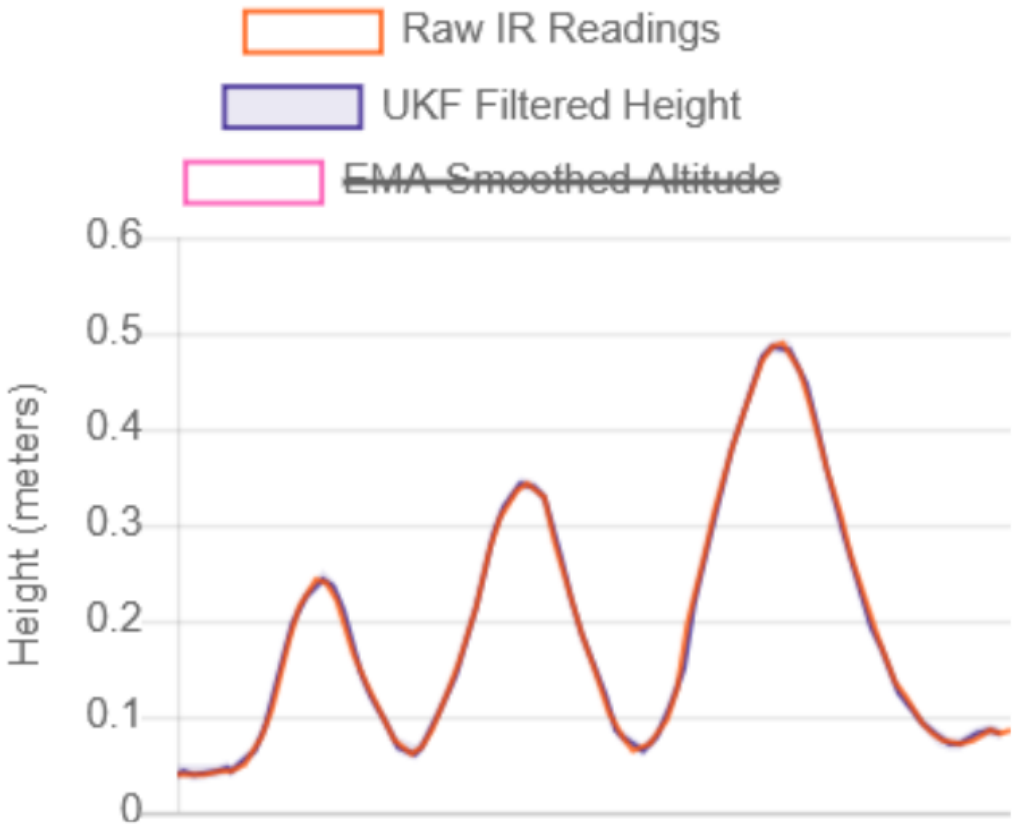
The predict step uses the control input to estimate the state at the current time. The update step corrects this state estimate based on the current measurement readings. It is reasonable to do multiple consecutive predict steps because the state is constantly changing with time and the filter can make a new prediction of the state with every new control input without needing to correct for measurements. If multiple update steps are done without prediction steps in between, the state estimate will not factor in control inputs that occurred in between update steps. Subsequent update steps add further corrections to an old state prediction instead of correcting a current state prediction. With multiple consecutive update steps, the stored state

estimate is outdated and the measurement corrections do not help to make the stored estimate a more accurate assessment of the current state.

### 2.3 Comparison of UKF to EMA Filter

EMA filters tend to lag as direction changes because its estimate is heavily influenced by previous values. Kalman filters, on the other hand, predict the next state before receiving a measurement, meaning they are subject to less lag. The Kalman filter also estimates error covariance, which provides the ability to assess uncertainty and the reliability of state estimates. In experiments with the drone, the raw IR readings were relatively noise free. This makes it apparent that the UKF smoothing more closely follows the actual IR readings with less lag. The UKF estimate is also smoother and does not have as many sharp points, whereas the EMA filter includes sharp changes that are not reflected in the original IR readings.




Raw IR Readings
UKF Filtered Height
EMA-Smoothed Altitude
Height (meters)
0.6
0.5
0.4
0.3
0.2
0.1
0